

# 1. Introduction

## 1.1 Context and Motivation

The software industry widely uses microservices to build scalable and modular applications. However, this distributed architectural style introduces severe complexity in system reliability. When an application is split across multiple independent services, activities such as fault detection, fault tolerance, and root cause analysis become significantly more challenging than in traditional monolithic systems (Silva et al., 2022). Developers and other stakeholders therefore face two common problems: identifying possible faults, and choosing the right tools to observe these faults. The volume and variety of available options can be overwhelming. As a consequence, development teams often spend excessive time on debugging, driving up operational costs and preventing organisations from realising the full potential of microservices.

## 1.2 Purpose

The purpose of this project was to analyse and combine the findings of two recent works – Towards a Fault Taxonomy for Microservices-Based Applications (Silva et al., 2022) and Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms (Faseeha et al., 2025) – and to offer an applied Proof-of-Concept that (1) bridges the gap between highly theoretical findings and practice, and (2) associates specific faults with specific observability tools.

## 1.3 Approach

The project selected specific faults from the taxonomy proposed by Silva et al. (2022) and mapped them to appropriate observability tools according to the three observability pillars: logs, metrics, and traces, as defined by Faseeha et al. (2025). A small microservices environment was then developed, and through fault injection techniques, specific faults were deliberately introduced into these microservices. This allowed for a practical evaluation of how industry-standard observability tools (Prometheus and Jaeger) are configured, how they operate, and how effectively they detect and isolate the root cause of these faults.

## 1.4 Research Questions

The project was guided by the following research questions:

**RQ1:** How can theoretical fault taxonomies in microservices be systematically mapped to the functional purposes of modern observability tools (metrics, traces, logs)?

**RQ2:** How effective are specific observability tools (e.g., Prometheus, Jaeger) at practically detecting and diagnosing injected faults within a controlled microservices environment?

---

## 2. Background and Related Work

## 2.1 Microservices Architecture

Modern software systems have transitioned from monolithic architectures to microservices to achieve independent deployability and scalability. However, this shift introduces a new complexity level. In a microservices environment, a single user request may traverse dozens of services over a network, and a failure in one service can lead to cascading effects across the entire system. Traditional debugging methods are insufficient in this setting, as the state of the application is fragmented across multiple isolated containers and network hops.

## 2.2 A Taxonomy of Microservices Faults

Silva et al. (2022) conducted a systematic literature review resulting in a comprehensive taxonomy of 117 microservice-specific faults. These faults are classified across six non-functional requirement categories: security, performance, reliability, maintainability, compatibility, and functionality.

For this project, the performance and reliability categories were of particular interest. Performance faults, such as Long Response Time (PF20) and CPU Hog (PF31), are often difficult to detect because the affected service typically remains "up" while still failing to meet Service Level Objectives (SLOs). Reliability faults, such as Service Unavailable (RF12), manifest more abruptly as discrete failure events.

Notably, Silva et al. (2022) reported that CPU Hog was the single most-cited fault across the studies they reviewed (5 of 28 studies), with Memory Leak in second place (4 studies).

## 2.3 Observability in Microservices: Frameworks and Paradigms

Faseeha et al. (2025) provided a comprehensive review of observability frameworks designed for cloud-native, microservices-based architectures. The paper argued that while microservices offer scalability, they simultaneously introduce significant complexities which traditional monitoring approaches cannot adequately address.

The core of Faseeha et al.'s work is the organisation of observability into a thematic taxonomy based on three pillars: **logs, traces, and metrics**. Each pillar serves a distinct diagnostic purpose:

- **Metrics** provide aggregate, numerical insights into performance and resource utilisation (CPU, memory, throughput) and are well-suited to anomaly detection. Faseeha et al. (2025) illustrated this with a canonical example: if a server's CPU normally ranges between 20–40%, sudden use of 90% could be identified as an anomaly.
- **Traces** provide per-request structural views by decomposing requests into spans across services. Traces are best suited to root cause analysis, identifying the specific component or function responsible for a fault within a distributed call chain. Faseeha et al. (2025) specifically highlighted Jaeger as a tool that "enables detailed tracking of requests across services, helping to identify bottlenecks, optimise performance, and conduct effective root cause analysis."

- **Logs** capture discrete, descriptive events with rich textual context (exception types, stack traces, error messages). Logs are well-suited to application health diagnostics: understanding why a specific failure occurred, beyond the fact that it occurred.

This three-pillar framework provided the conceptual foundation for the mapping exercise undertaken in this project.

---

### 3. Methodology

#### 3.1 Selection of Faults

From the 117 faults identified by Silva et al. (2022), three were selected for this Proof-of-Concept. The selection was guided by three criteria:

1. The fault had to be possible to inject in a Python-based microservices environment without requiring extra infrastructure.
2. The fault had to be relevant to modern microservices and the kinds of failures they typically have.
3. The three faults together had to cover all three observability pillars, so that the project could exercise each pillar within a single Proof-of-Concept.

The selected faults were:

- **PF31 (CPU Hog):** A performance fault in Silva et al.'s (2022) Resource Utilization subcategory, where a service uses too much processing power. Selected as a clear example of a resource-consumption fault, and also the most-cited fault in Silva et al.'s (2022) review.
- **PF20 (Long Response Time):** A performance fault in Silva et al.'s (2022) Temporal Behavior subcategory, where requests take too long to complete. Selected because the slowness can come from a specific function inside the call chain.
- **RF12 (Service Unavailable):** A reliability fault where a service stops responding and returns an error. Selected because it represents a one-off failure event, different from the gradual slowdowns of the two performance faults above.

Together, these three faults cover two of Silva et al.'s (2022) six categories (performance and reliability) and match the three observability pillars defined by Faseeha et al. (2025), as explained in the next section.

#### 3.2 Mapping Logic and Theoretical Justification

To connect Silva et al.'s (2022) fault taxonomy with Faseeha et al.'s (2025) observability framework, each fault was paired with the observability pillar that fits it best. The pairing was based on the kind of evidence each fault leaves behind in a running system: a numerical change, a slow component in a request, or a specific failure event.

It is important to be clear about what this mapping does and does not claim. It does not claim that a fault can only be seen through its mapped pillar. Modern observability stacks are

overlapping: a slow request shows up in metrics (higher latency), in traces (a long span), and in logs (timestamped request entries). The mapping only reflects which pillar gives the best diagnostic information for each fault type, and which of Faseeha et al.'s (2025) four monitoring purposes (Performance Analysis, Failure Root Cause Analysis, Anomaly Detection, and Overall Application Health) fits each fault best.

### **Metrics for Anomaly Detection — PF31 (CPU Hog)**

CPU Hog is a resource-consumption fault: it causes a clear change in numerical counters (process CPU time, request latency) while the service keeps running. Faseeha et al. (2025) defined Anomaly Detection as the process of finding patterns that fall outside normal behaviour, and gave a CPU example that fits PF31 directly: "if a server's CPU normally ranges between 20–40%, sudden use of 90% could be identified as an anomaly" (Faseeha et al., 2025).

Detecting this kind of fault with metrics does not require knowing in advance where in the code the problem is. A Prometheus instance scraping `process_cpu_seconds_total` will pick up the change no matter which function caused it. This is what makes metrics the natural fit for resource-consumption faults.

### **Traces for Performance Analysis — PF20 (Long Response Time)**

A slow request is a different kind of problem from a resource shift. The slowness lives somewhere in the call chain, possibly across several services and many functions. Aggregate latency metrics can show that the system is slow but cannot show where the slowness comes from. Faseeha et al. (2025) defined Performance Analysis as the continuous monitoring of how services handle requests, with response time as one of the main things to watch: by tracking these times, an organisation can "observe which services are performing slowly and thus need optimisation" (Faseeha et al., 2025).

While metrics can flag that a service is slow overall, distributed tracing answers the more specific question: which component is the slow one? A trace breaks each request into spans with timing information, so the slow part can be spotted directly inside the request. This is what makes traces the natural fit for temporal-behaviour faults.

### **Logs for Failure Root Cause Analysis — RF12 (Service Unavailable)**

A failed request is a one-off event: it happens at a specific moment, has a specific cause, and produces a specific artefact (an HTTP error, a logged exception). Faseeha et al. (2025) defined Failure Root Cause Analysis as the detailed investigation of errors, exceptions, and the events that lead to a failure and named logs as the main source for this work: an organisation can "seek to know where and why a certain failure happened by gathering logs recording the occurrences of errors and exceptions", which provide "information on the nature of the error, the components involved, and the context under which the issue occurred" (Faseeha et al., 2025).

Metrics can count failures (a spike in 5xx responses) and traces can show that a span has an error, but the textual detail needed to understand the failure (which are the exception

type, the message, the stack trace) sits most naturally in the log record. This is what makes logs the natural fit for event-based reliability faults.

Summary mapping

The following table summarises the theoretical mapping that served as the foundation for the empirical work:

| Fault Code (Silva) | Fault Category | Most Natural Pillar (Faseeha) | Targeted Tool    | Diagnostic Purpose          |
|--------------------|----------------|-------------------------------|------------------|-----------------------------|
| PF31               | Performance    | Metrics                       | Prometheus       | Anomaly Detection           |
| PF20               | Performance    | Traces                        | Jaeger           | Performance Analysis        |
| RF12               | Reliability    | Logs                          | Container stdout | Failure Root Cause Analysis |

It is worth noting why container stdout was chosen as the logging surface for RF12 rather than a centralised aggregation system such as Loki. For the scope of this Proof-of-Concept, raw container logs accessed via `docker compose logs` were sufficient to demonstrate the diagnostic content carried by log records, but in a production deployment this stream of logs would typically be ingested by a centralised log aggregator

Relevance of this mapping

For the reader, the practical value of this mapping is that it gives a starting point for matching a class of fault to a class of tool. A team facing an unfamiliar performance issue can ask "is this a resource problem or a slow-component problem?" and use the answer to decide where to look first, Prometheus dashboards or Jaeger traces. This narrows the search space compared to checking all three pillars at once.

The mapping is not meant as a strict rule. As Chapter 5 will show, the experimental results both confirmed the natural fits above and revealed cases where more than one pillar showed the same fault from a different angle. These overlaps do not contradict the framework but they refine it.

---

## 4. Experimental Setup and Implementation

### 4.1 System Architecture

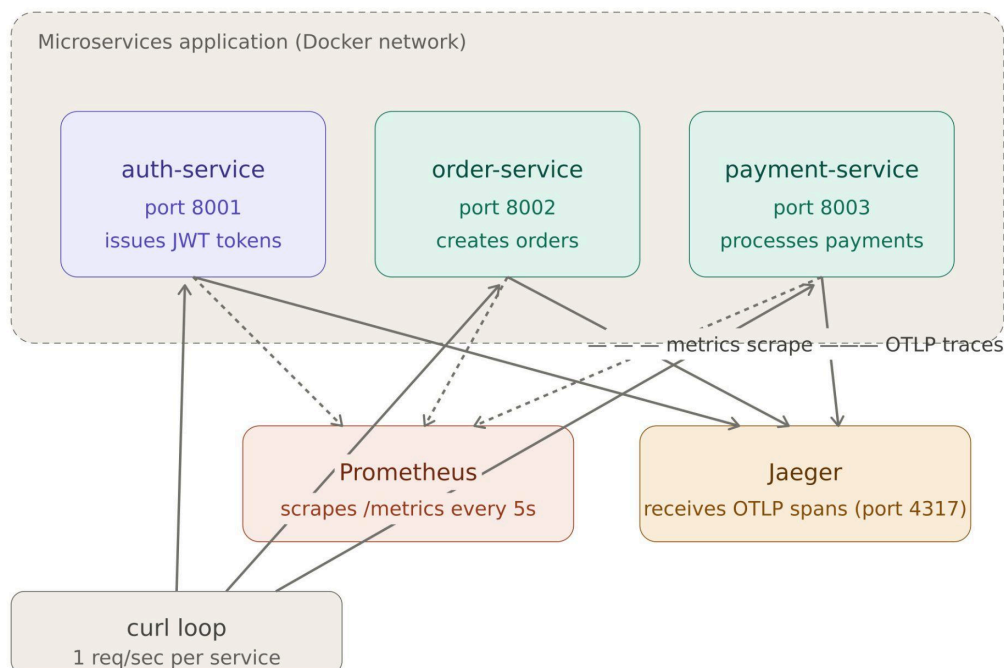
A minimal three-service application was developed to simulate a small-scale microservices environment. The services were intentionally kept simple, so that the experimental results would primarily reflect the behaviour of the injected faults rather than incidental complexity in the application logic. Each service was implemented in Python using the FastAPI web framework and packaged as an independent Docker container.

The three services were:

- **auth-service** (port 8001): Issues JWT tokens via a `POST /auth/login` endpoint. Accepts any username and password and returns a signed token valid for 24 hours.
- **order-service** (port 8002): Accepts authenticated `POST /orders` requests carrying a JWT in the `Authorization` header. Validates the token, generates an order ID, and records the order in an in-memory dictionary.
- **payment-service** (port 8003): Accepts authenticated `POST /payments` requests, validates the token, generates a payment ID, and records the payment in memory.

The services communicated over an isolated Docker bridge network (`research-network`), and the entire stack was orchestrated via Docker Compose.

The chosen design intentionally avoided shared state (databases, caches, message queues) to ensure that observed behaviours could be attributed unambiguously to the injected faults rather than to incidental infrastructure interactions.



**Figure 4.1.** System architecture. Three FastAPI microservices (auth, order, payment) communicate over an isolated Docker network. Each service is instrumented to export Prometheus-format metrics (scraped every 5 seconds, dashed lines) and OpenTelemetry traces via OTLP (solid lines to Jaeger). Traffic is generated by `curl` loops at one request per second.

## 4.2 Observability Stack

Two industry-standard observability tools were integrated alongside the application:

- **Prometheus** (port 9090): A pull-based metrics collection and time-series database, configured to scrape each service's `/metrics` endpoint every five seconds. Metric storage was persisted via a Docker volume (`prometheus_data`).
- **Jaeger** (port 16686): A distributed tracing system, deployed as the official `jaegertracing/all-in-one` image.

Instrumentation was implemented through the OpenTelemetry SDK and the `prometheus-fastapi-instrumentator` library. Both were initialised by a single helper function (`setup_observability`) called from each service at startup. The function created an OpenTelemetry `TracerProvider` configured with the service name as a resource attribute, registered a `BatchSpanProcessor` exporting via OTLP gRPC to Jaeger on port 4317, instrumented the FastAPI application for automatic per-request span creation, and registered Prometheus instrumentation that exposed standard process metrics and HTTP histograms at `/metrics`.

In addition to the auto-generated spans for each HTTP request, the service code wrapped each fault-injection function in an explicit span. This was a deliberate instrumentation choice: it allowed Jaeger to localise faults at the function level for the purposes of the experiment.

## 4.3 Fault Injection Mechanism

Faults were injected through environmental variables read at request handling time. A request entered the relevant injection section only when the corresponding variable was set to `"true"` in the service container at start-up. This design has three properties relevant to the experiment:

1. **Reproducibility:** A fault's presence is determined entirely by the service's configuration at start-up. There is no probabilistic or stateful component.
2. **Isolation:** Each fault affects only the service whose container holds the activating variable. Faults can be enabled and disabled independently per service.
3. **Minimal intrusiveness:** No traffic was rerouted or proxied. The fault is part of the request handling code path itself.

The three injected faults were implemented in `services/faults.py` as follows.

**PF31 — CPU Hog.** A synchronous arithmetic loop performing twenty million additions, executed inline within the request handler:

```
def cpu_hog():
    logger.info("PF31: INJECTED CPU HOG FAULT")
    count = 0
    for i in range(2 * 10**7):
        count += i
```

The loop's sole effect is to consume CPU cycles within the request lifetime. Because the loop runs in the same coroutine as the request handler, it directly extends the request's duration in addition to consuming CPU time.

**PF20 — Long Response Time.** An asynchronous sleep of eight seconds:

```
async def long_response_time():
    logger.info("PF20: INJECTED LONG RESPONSE TIME FAULT")
    await asyncio.sleep(8)
```

`asyncio.sleep` suspends only the affected coroutine, allowing other requests to proceed in parallel. This more accurately models a real I/O-bound dependency (like a slow database query or external API call) where the service itself remains responsive but a particular request is stalled awaiting an external response.

**RF12 — Service Unavailable.** An `HTTPException` raised immediately upon entry to the request handler, returning a 503 status code:

```
def service_unavailable():
    logger.info("RF12: INJECTED SERVICE UNAVAILABLE FAULT")
    raise HTTPException(status_code=503, detail="Service Unavailable")
```

When the affected service is `auth-service`, every login attempt fails with 503, preventing new tokens from being issued.

The fault-injection sections were guarded by environmental variable checks at the top of each handler. For example, the order handler began with:

```
if os.getenv("INJECT_CPU_HOG") == "true":
    with tracer.start_as_current_span("cpu_hog"):
        cpu_hog()
```

## 4.4 Experimental Procedure

For each fault, the experiment compared a baseline period (no faults active) with an injection period (one fault active in one service). Traffic was generated using `curl` loops issuing one request per second, run in separate terminal sessions for each service. A single JWT token

was acquired at the start of each experimental session via the auth-service Swagger UI and reused throughout.

The procedure for each fault followed the same pattern:

1. **Baseline phase** ( $\geq 3$  minutes). With all `INJECT_*` environment variables set to `false`, traffic loops were started against `order-service` and `payment-service`. Prometheus and Jaeger accumulated baseline metrics and traces.
2. **Transition**. The traffic loop for the affected service was stopped. The relevant `INJECT_*` variable was set to `true` in the project's `.env` file. The single affected container was recreated using `docker compose up -d --force-recreate --no-deps <service>`. The `--no-deps` flag was important: it prevented Docker Compose from also recreating dependencies (notably Jaeger), preserving the in-memory baseline traces accumulated during the baseline phase.
3. **Injection phase** ( $\geq 3$  minutes). The traffic loop was restarted against the now-injected service. Prometheus and Jaeger continued capturing data, now reflecting the injected fault.
4. **Capture**. Screenshots were taken from the Prometheus and Jaeger UIs covering the full window of the experiment, allowing baseline and injection behaviour to be compared within a single image. Container logs were captured separately via `docker compose logs` for the relevant service.

For RF12, an additional consideration applied: since auth-service became unable to issue tokens during injection, the JWT token used by the order- and payment-service loops had to be obtained before injection began. The 24-hour token validity ensured this was unproblematic.

A summary of the parameters used across all three experiments is provided below.

| Parameter                                   | Value                                                 |
|---------------------------------------------|-------------------------------------------------------|
| Traffic generation                          | <code>curl</code> loops, 1 request/second per service |
| Token validity                              | 1440 minutes (24 hours)                               |
| Baseline phase duration                     | $\geq 3$ minutes                                      |
| Injection phase duration                    | $\geq 3$ minutes                                      |
| Prometheus scrape interval                  | 5 seconds                                             |
| Rate window for <code>rate()</code> queries | 1 minute                                              |
| Histogram quantile windows                  | 1 minute                                              |
| Container recreation                        | <code>--force-recreate --no-deps</code>               |

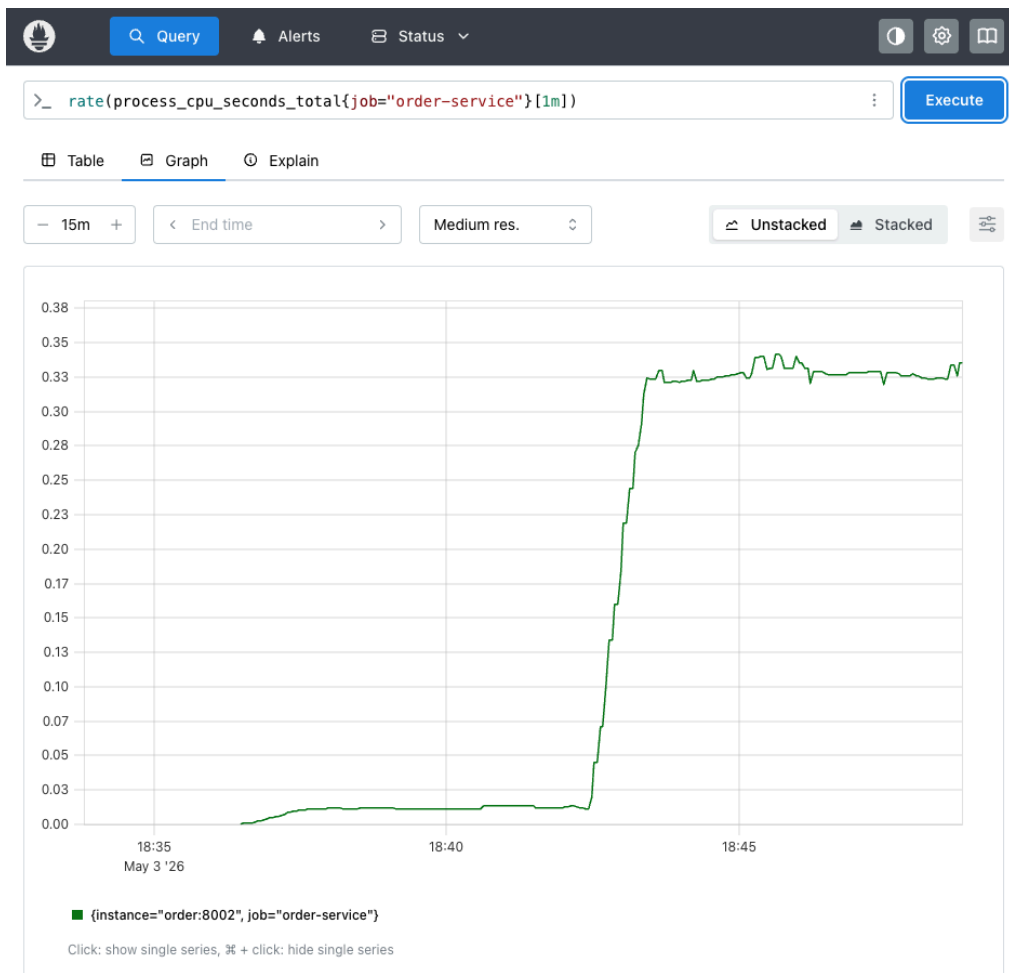
## 5. Results

### 5.1 PF31 — CPU Hog

PF31 was injected into `order-service`. The fault inserted a tight arithmetic loop into every `POST /orders` request handler (Section 4.3), consuming CPU cycles within the request lifetime. The expected diagnostic signature was a sustained shift in process-level CPU consumption, accompanied by elevated request latency.

#### Primary detection: process CPU rate

Figure 5.1 shows the rate of `process_cpu_seconds_total` for the order service, evaluated over a one-minute rolling window. During the baseline phase, CPU consumption sat near zero (approximately 0.01) indicating that less than 1% of one CPU core was active on average. Following the injection at approximately 18:42, CPU consumption rose sharply to a steady plateau of approximately 0.33, indicating sustained use of roughly one third of a CPU core.



[Figure 5.1]

`process_cpu_seconds_total` is a counter tracking the cumulative CPU-seconds consumed by the process since it started. The counter grows only while the process actively executes on a CPU core. Wrapping it in `rate(...[1m])` produces the per-second rate of

growth – averaged over the last 60 seconds of data – expressed as a fraction of one core: 0.01 indicates near-idle, 0.5 indicates half a core, 1.0 indicates a fully saturated core.

CPU consumption rose immediately upon injection from a near-idle baseline (~0.01) to a sustained plateau (~0.33). The exact magnitude is determined by the size of the injected loop and the request rate, and is not itself a meaningful finding. What matters is that the fault produces a clear, sustained shift away from baseline behaviour, which is exactly what metrics-based anomaly detection is designed to catch

The one-minute ramp visible at the transition reflects the rate function's rolling window populating with injection-period data, not a gradual onset of the fault.

### User-facing effect: request latency

Figure 5.2 shows the 50th-percentile latency of `POST /orders` requests. During baseline, the median request completed in approximately 50 milliseconds. Following injection, the median rose abruptly to approximately 330 milliseconds, a six-fold increase in typical user-perceived latency.

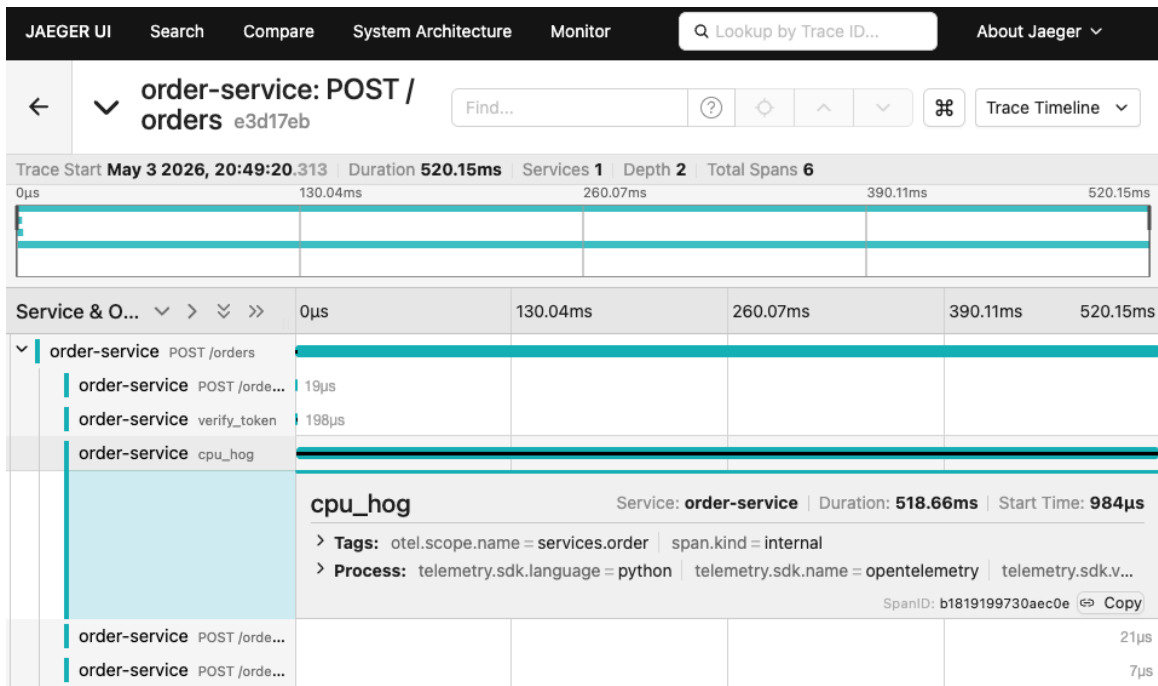


[Figure 5.2]

The latency increase is the user-visible effect of the CPU consumption shown in Figure 5.1. A request that was previously executed quickly now also performs the artificial computation.

Diagnostic localisation in Jaeger

Figure 5.4 shows a representative Jaeger trace captured during PF31 injection. The trace contains six spans: the root `POST /orders` span (520 milliseconds), the auto-generated `POST /orders http receive` span (19 microseconds), the explicitly instrumented `verify_token` span (198 microseconds), the explicitly instrumented `cpu_hog` span (518 milliseconds), and two `POST /orders http send` response spans (21 and 7 microseconds).



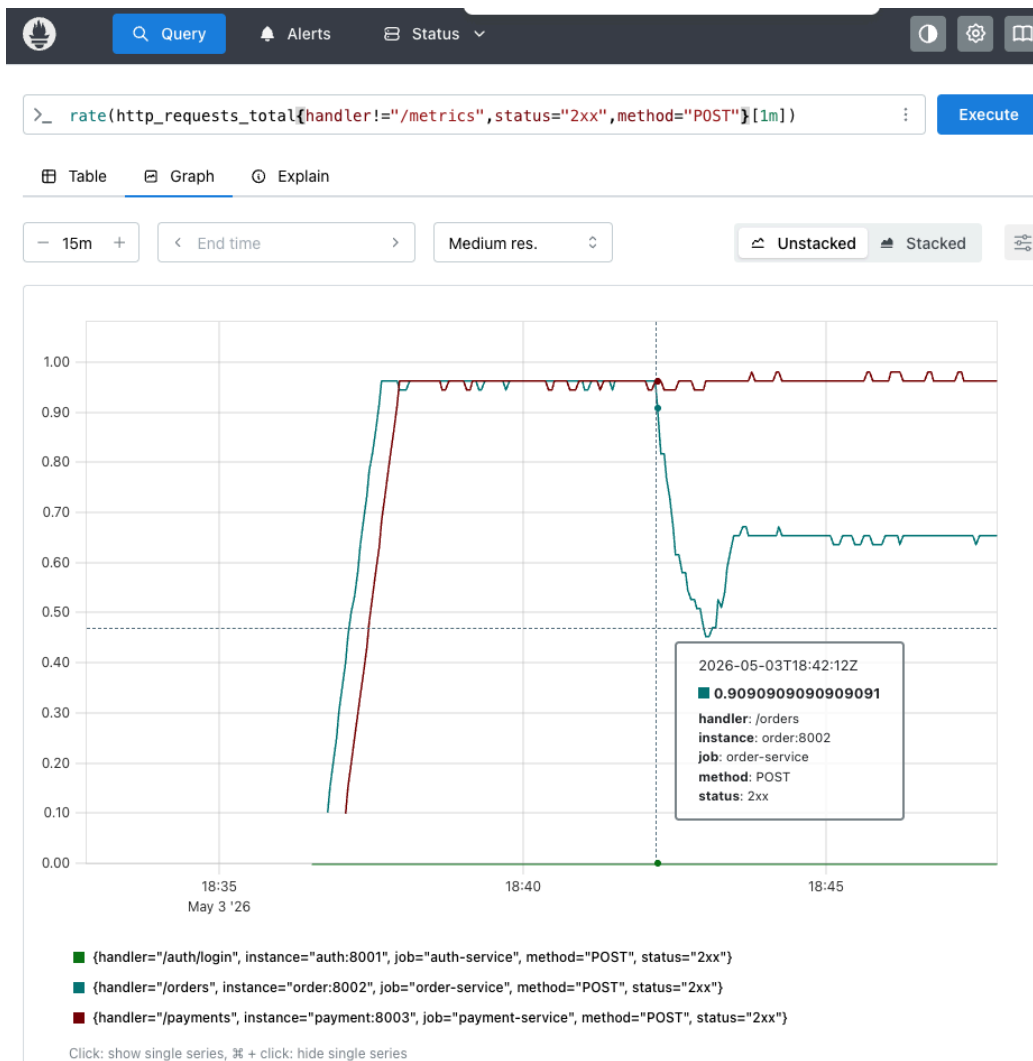
[Figure 5.4]

The `cpu_hog` span accounts for 99.7% of the trace's duration. Every other operation in the request is much faster.

It is worth noting that the `cpu_hog` function was deliberately wrapped in an OpenTelemetry span. Without it, Jaeger would have shown only the root `POST /orders` span as 520ms – with no internal breakdown – so the fact that Jaeger localised PF31 to a specific function is therefore a property of the instrumentation.

Fault containment

Figure 5.5 shows the success rate of `POST` requests across all three services. During the injection phase, the order-service rate dropped from approximately 0.96 to 0.71 requests per second. The auth-service and payment-service rates remained at their baseline levels throughout.



[Figure 5.5]

## Summary

Silva et al. (2022) classify CPU Hog as a performance fault in the resource-utilisation subcategory, i.e. faults where a service uses more resources than it should. Chapter 3 paired this category with Prometheus and the metrics pillar, because such faults show up as changes in resource counters, which is exactly what Faseeha et al (2025) anomaly detection use case is built for.

The experimental data confirmed this mapping. The injected fault produced a clear, sustained increase in `process_cpu_seconds_total`, detected by a single standard query against Prometheus's default process-level metrics. No extra configuration or prior knowledge of where the fault was happening was required.

For RQ1, the result supports the proposed mapping for resource-utilisation performance faults. For RQ2, Prometheus detected the fault effectively with very little effort.

Jaeger also localised the fault to the explicitly instrumented `cpu_hog` function, more detailed than detection alone, but only possible because the function had been wrapped in a span beforehand.

## 5.2 PF20 — Long Response Time

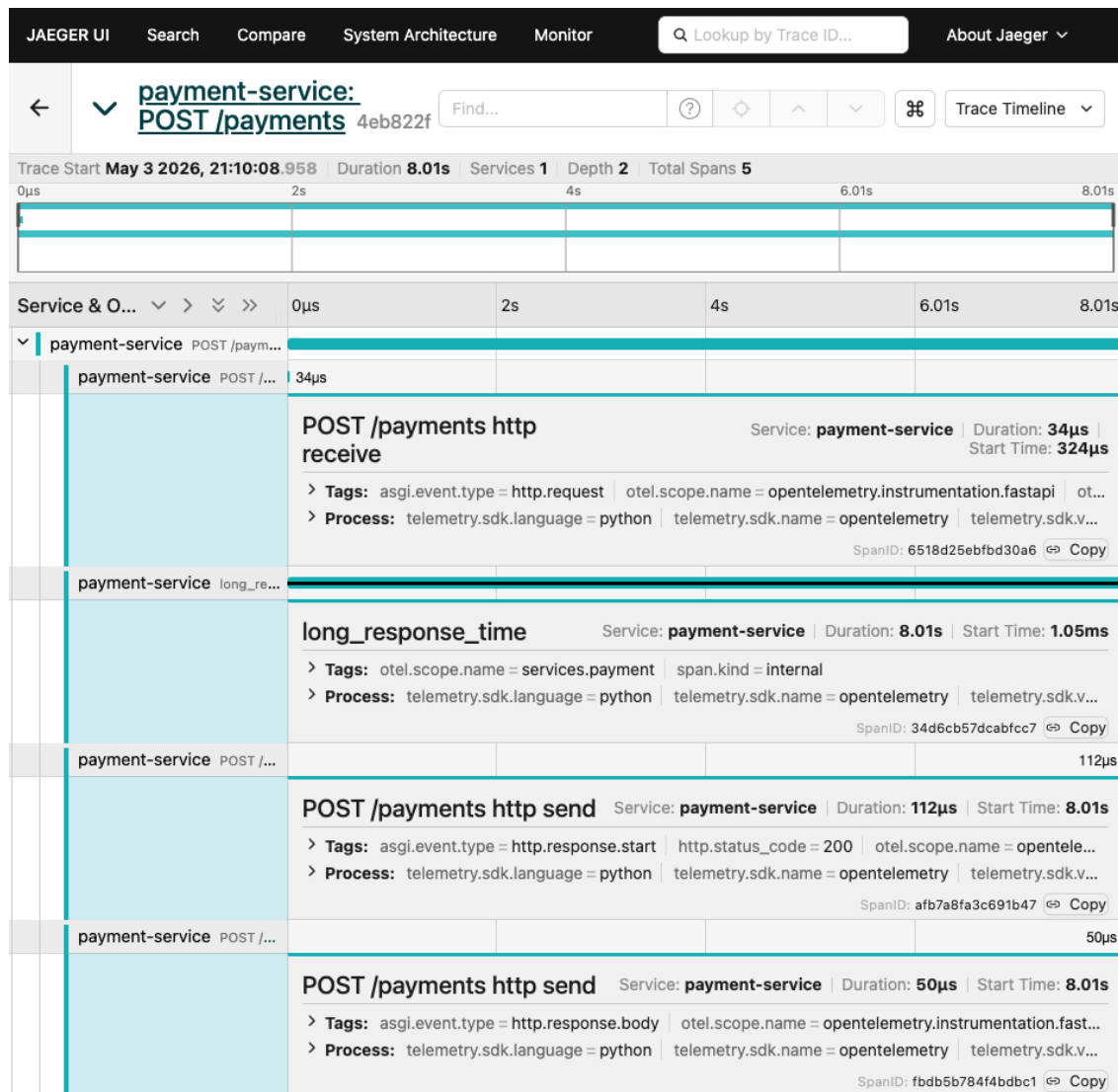
PF20 was injected into `payment-service`. The fault inserted an `await asyncio.sleep(8)` into every `POST /payments` request handler. The expected diagnostic signature was higher request latency without a corresponding rise in CPU consumption.

### Primary detection: distributed trace breakdown

Figure 5.6 shows a Jaeger trace captured during PF20 injection. The trace contains five spans: the root `POST /payments` span (8.01 seconds), the auto-generated `POST /payments http receive` span (34 microseconds), the explicitly instrumented `long_response_time` span (8.01 seconds, starting 1 millisecond into the request), and two `POST /payments http send` response spans (112 and 50 microseconds).

The `long_response_time` span takes up the entire trace duration. As with PF31's `cpu_hog` span, this only works because the function was wrapped in a span deliberately, as described in Section 4.2. Without that, the trace would show only an 8-second root span with no internal detail.

The trace does not just say that the payment endpoint is slow but it also shows exactly which step in the request is slow. This matches Faseeha et al.'s (2025) description of Performance Analysis, where distributed tracing is used to identify which services or components are slow and need attention.



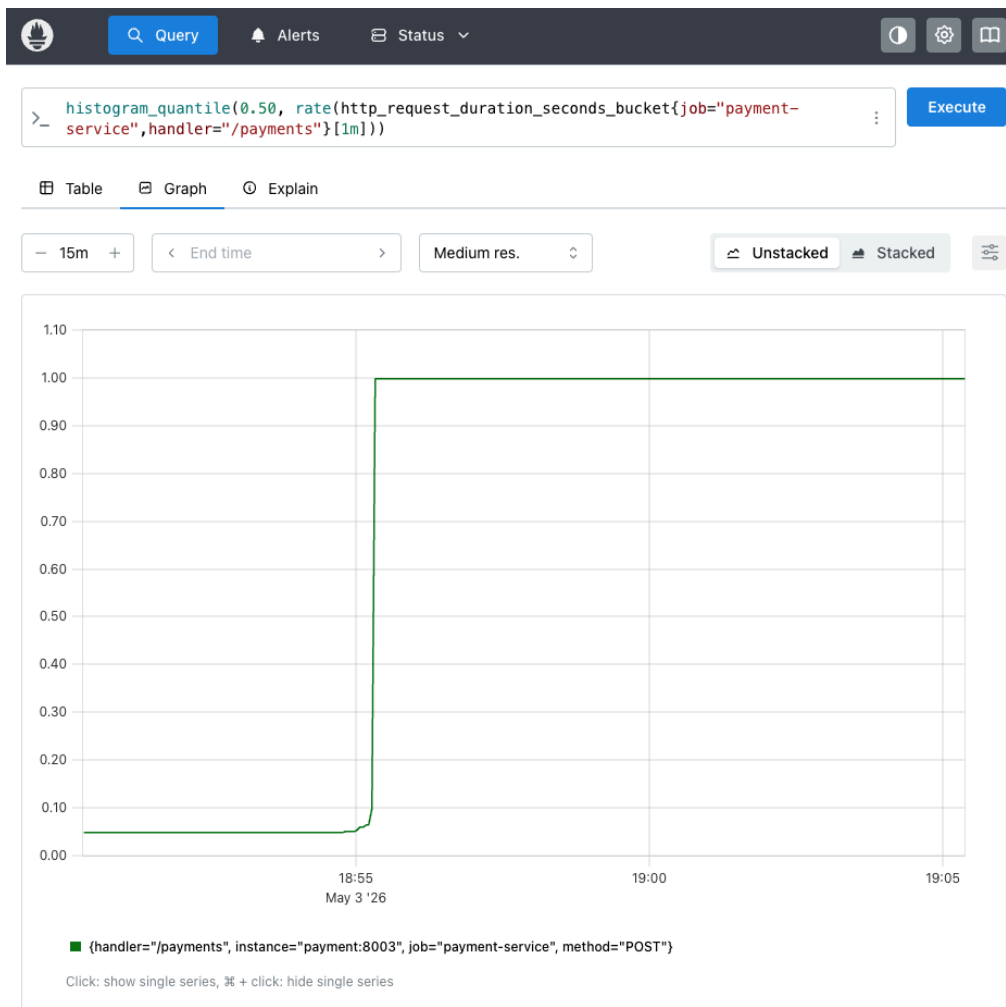
[Figure 5.6]

## Aggregate latency in Prometheus

Figure 5.7 shows the 50th-percentile latency of `POST /payments` requests over the same window. During the baseline phase, the median request completed in around 50 milliseconds. After injection, the median rose sharply but plateaued at exactly 1.0 second.

The 1.0-second plateau is not the real latency. It comes from how the histogram is configured. The `prometheus-fastapi-instrumentator` library sets the largest finite bucket at `le="1.0"`. Any request taking longer than one second falls into the `le="+Inf"` bucket, which `histogram_quantile` cannot use to estimate a numerical value. The result is capped at the largest finite bucket.

The actual median during injection was around 8 seconds, as Jaeger measured directly (Figure 5.6). For detecting the fault, the cap on p50 was still enough: the value clearly jumped above baseline. For knowing how slow the requests were, only Jaeger gave an accurate answer.



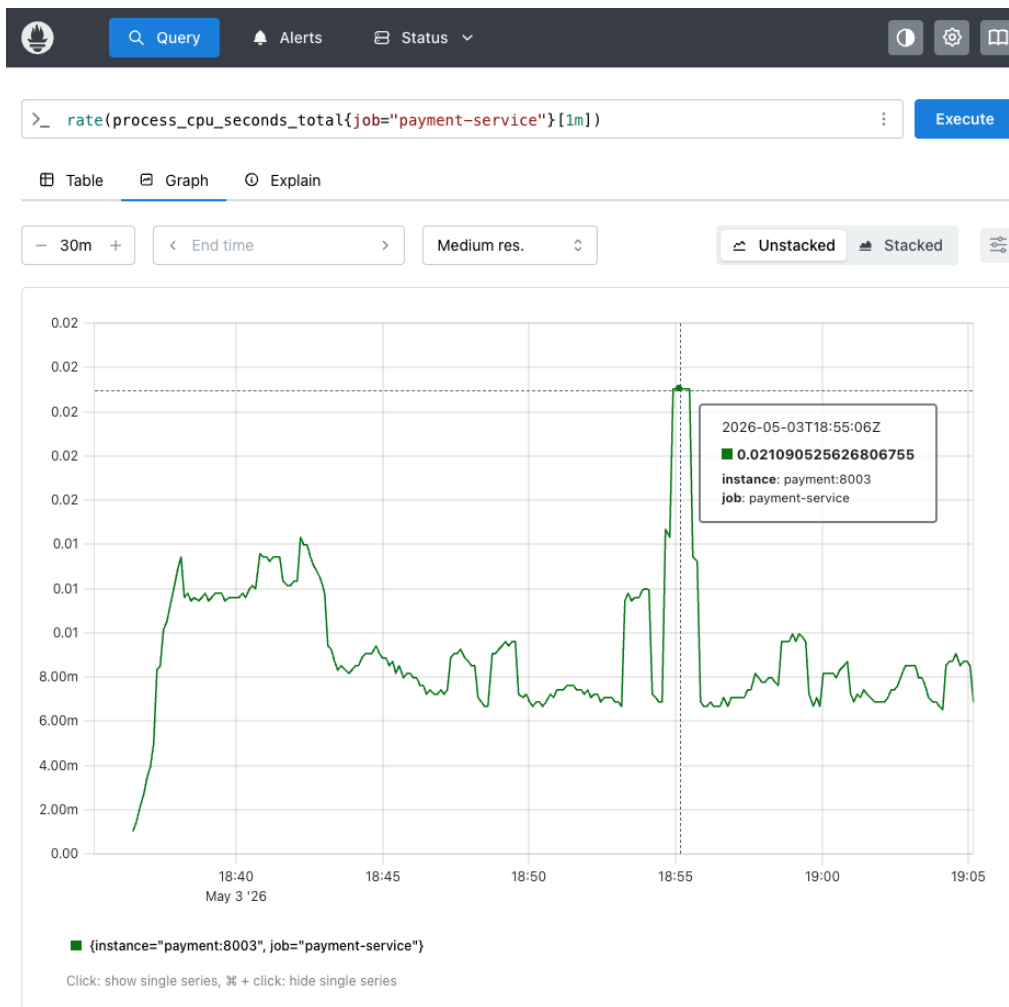
[Figure 5.7]

### CPU consumption: the key differentiator

Figure 5.8 shows the rate of `process_cpu_seconds_total` for the payment service across the same window.

Unlike PF31 (Figure 5.1), CPU usage did not rise during injection. Throughout both phases, the payment-service process used less than 1% of one CPU core on average. Even though each request now took around 8 seconds, the process was not doing any work during that time (it was waiting on `asyncio.sleep`) which suspends the coroutine without using CPU.

This is the most useful diagnostic result of the experiment. It shows that metrics can tell apart CPU-bound faults from wait-bound faults, even when both cause higher latency. Looking only at the latency histogram (Figure 5.7), an operator would not know whether the slowness came from heavy computation or from waiting on something external. Adding the CPU rate metric answers that question before any trace investigation is needed.



[Figure 5.8]

## Summary

Silva et al. (2022) classify Long Response Time as a performance fault in the temporal-behaviour subcategory (faults where requests take too long to complete). Chapter 3 paired this category with Jaeger and the traces pillar, because finding where in a distributed call chain a delay comes from is the kind of work Faseeha et al.'s (2025) Performance Analysis use case describes: tracking response times to "observe which services are performing slowly and thus need optimisation".

The experimental data confirmed this mapping. Jaeger localised the eight-second delay directly to the `long_response_time` span, while every other span completed in microseconds. The trace did not just show that the request was slow, but it also showed where, in the request, the slowness lived. Prometheus also detected the latency increase but capped the reported value at one second because of the default histogram buckets, and could not point to the offending function.

For RQ1, the result supports the proposed mapping for temporal-behaviour performance faults. For RQ2, Jaeger diagnosed the fault precisely; Prometheus detected it but with reduced fidelity.

A useful side finding also emerged: the CPU rate stayed flat throughout, telling PF20 apart from PF31 at the metric level even though both caused higher latency. This shows that metrics offer more than detection, they can also classify what kind of fault is occurring.

### 5.3 RF12 — Service Unavailable

RF12 was injected into `auth-service`. The fault made every `POST /auth/login` request immediately raise an `HTTPException` with status code 503, preventing new tokens from being issued. The expected diagnostic signature was a series of one-off failure events, each carrying explicit error context.

#### Primary diagnosis: container logs

Figure 5.10 shows a sample of `auth-service` container logs captured during the injection phase via `docker compose logs --tail=80 auth`.

Two log streams are visible. The first is the log line emitted by the fault function itself (`RF12: INJECTED SERVICE UNAVAILABLE FAULT`), tagged with the logger name `fault injections` and a timestamp. The second is the standard FastAPI access log, recording the matching HTTP response: `"POST /auth/login HTTP/1.1" 503 Service Unavailable`.

This is the kind of diagnostic content that no other pillar provides directly. A metrics dashboard would record the rate of 5xx responses but not the application-level reason. A trace would record that the request errored but not the textual detail.

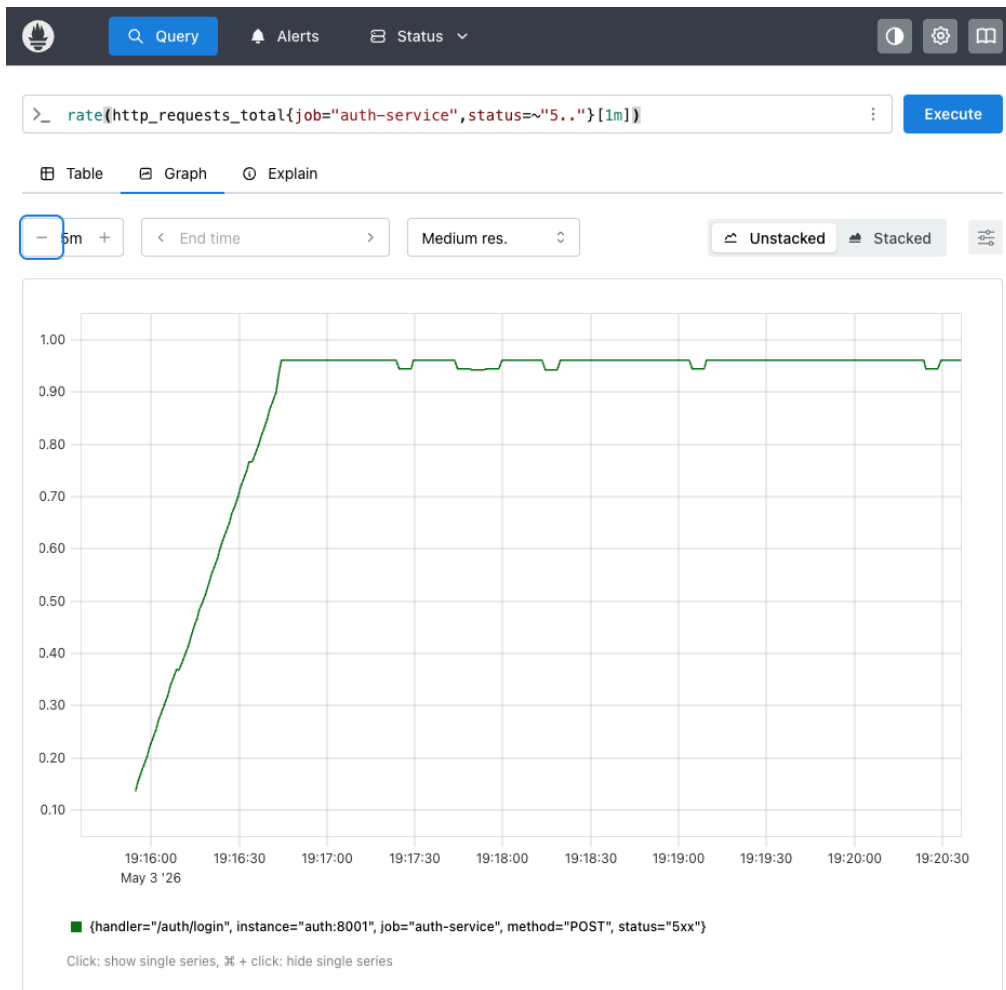
```
(rpvenv) → research-project-refactored git:(main) x docker compose logs --tail=80 auth
auth-service | INFO:      192.168.65.1:36287 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | 2026-05-03 19:18:32,456 | INFO      | fault injections | RF12: INJECTED SERVICE UNAVAILABLE FAULT
auth-service | INFO:      192.168.65.1:26680 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | 2026-05-03 19:18:33,482 | INFO      | fault injections | RF12: INJECTED SERVICE UNAVAILABLE FAULT
auth-service | INFO:      192.168.65.1:34670 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | INFO:      172.18.0.6:59690 - "GET /metrics HTTP/1.1" 200 OK
auth-service | 2026-05-03 19:18:34,506 | INFO      | fault injections | RF12: INJECTED SERVICE UNAVAILABLE FAULT
auth-service | INFO:      192.168.65.1:35762 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | 2026-05-03 19:18:35,536 | INFO      | fault injections | RF12: INJECTED SERVICE UNAVAILABLE FAULT
auth-service | INFO:      192.168.65.1:52537 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | 2026-05-03 19:18:36,565 | INFO      | fault injections | RF12: INJECTED SERVICE UNAVAILABLE FAULT
auth-service | INFO:      192.168.65.1:42226 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | 2026-05-03 19:18:37,587 | INFO      | fault injections | RF12: INJECTED SERVICE UNAVAILABLE FAULT
auth-service | INFO:      192.168.65.1:25029 - "POST /auth/login HTTP/1.1" 503 Service Unavailable
auth-service | INFO:      172.18.0.6:56740 - "GET /metrics HTTP/1.1" 200 OK
```

[Figure 5.10]

#### Secondary detection: Prometheus 5xx rate

Figure 5.12 shows the rate of HTTP responses from `auth-service` matching the `5..` status pattern.

During the baseline phase the rate sat at zero (no failures occurring); after injection it ramped up to about 0.96 requests per second, matching the `auth` loop's request rate. The metric shows that the requests to the service are failing, but does not give descriptive context on what the failure is and why it is failing.



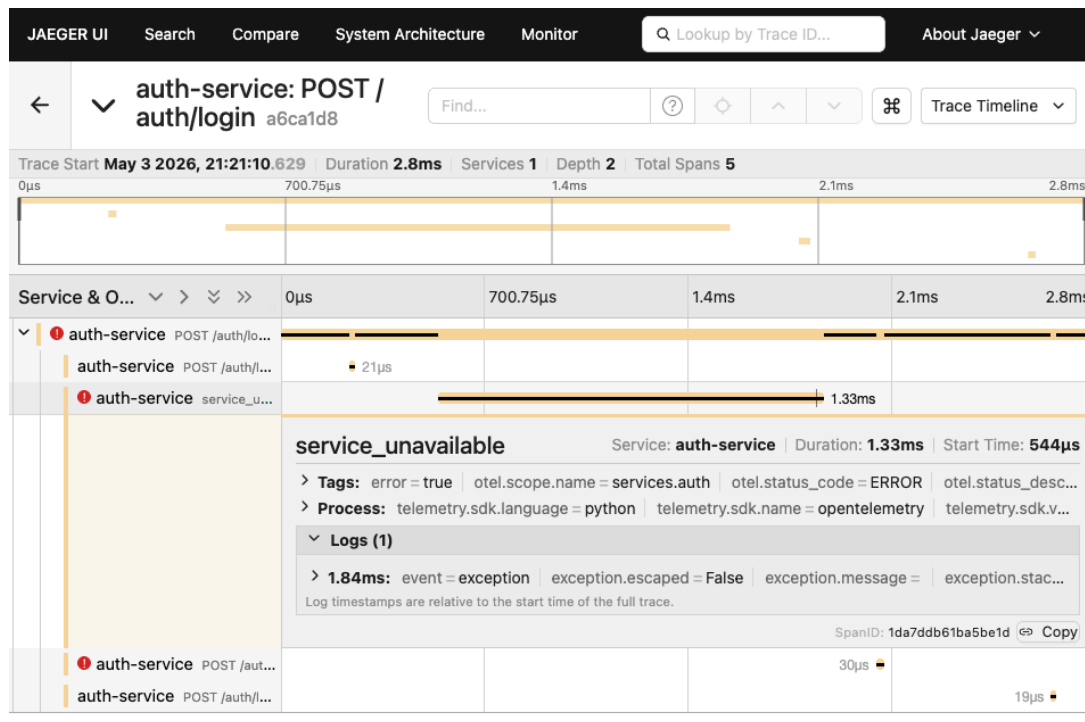
[Figure 5.12]

### A subtlety: exception capture in Jaeger

Figure 5.13 shows a failed Jaeger trace from auth-service during injection.

The `POST /auth/login` operation failed in 2.8 milliseconds (faster than a successful baseline login) since the fault function raises immediately rather than completing the login workflow. The trace also shows the explicitly instrumented `service_unavailable` span, which carries an exception event with the exception type, message, and stack trace, captured automatically by the OpenTelemetry instrumentation when the `HTTPException` was raised inside the span.

The trace also includes metadata tags from the OpenTelemetry SDK itself.



[Figure 5.13: Jaeger trace breakdown]

## Summary

Silva et al. (2022) classify Service Unavailable as a reliability fault, a one-off failure event where a service stops responding correctly. Chapter 3 paired this category with logs and the Failure Root Cause Analysis use case from Faseeha et al. (2025), which describes the investigation of errors and exceptions to find the cause of a failure.

The experimental data confirmed this mapping. Auth-service logs carried both the application-level fault marker (RF12: INJECTED SERVICE UNAVAILABLE FAULT) and the matching 503 access entries with explicit textual context, exactly the kind of evidence Faseeha's RCA use case calls for.

For RQ1, the result supports the proposed mapping for event-based reliability faults. For RQ2, logs diagnosed the fault directly with full descriptive content; Prometheus detected it but provided no diagnostic context.

It is worth mentioning, that the `service_unavailable` function was wrapped in an explicit OpenTelemetry span, and Jaeger therefore localised the failure to that exact span, pointing to the precise function in the request path responsible for the 503. As with PF31's `cpu_hog` and PF20's `long_response_time`, this localisation depended on a deliberate instrumentation choice. In a real-world system where every meaningful function is wrapped in a span, the same pattern would let an operator pinpoint the exact location of any captured failure.

## 5.4 Findings across the experiments

The three fault experiments individually confirmed that each fault could be detected by its mapped pillar. Taken together, they also surfaced patterns common to all three cases, both confirming the framework from Chapter 3 and refining it

### **Detection is not the same as diagnosis**

Every pillar detected every fault to some extent: Prometheus saw PF20 as elevated latency, Jaeger saw PF31 as a slow trace, Prometheus saw RF12 as a 5xx spike. What differed was the diagnostic content each pillar carried. Metrics flagged that something was wrong but not what; traces showed which span was slow but not the system-wide rate; logs gave the exception text but not the aggregate view.

For the reader, the practical takeaway is to avoid treating the mapping as a strict one-to-one rule. A team facing an unfamiliar issue can ask "is this a resource shift, a slow component, or a failure event?" and use the answer to decide which pillar to consult first, but once a candidate cause is found, the other pillars often provide useful confirmation. The framework is a triage aid, not a partition.

### **Metrics provide classification, not only detection**

PF31 and PF20 both produced higher request latency. A latency dashboard alone would have flagged both as "the system is slow" without differentiation. The CPU rate metric resolved the ambiguity directly: PF31 elevated CPU usage; PF20 left it at baseline. The contrast distinguishes a CPU-bound fault from a wait-bound fault at the metric level, before any tracing investigation is needed.

### **Histogram bucket calibration is non-negotiable in production**

PF20 exposed a practical limitation of histogram-based latency reporting. The default `prometheus-fastapi-instrumentator` configuration sets the largest finite bucket at `le="1.0"`. Requests longer than one second fall into `+Inf`, which `histogram_quantile` cannot interpolate. The result: any latency over one second is reported as exactly 1.0s.

For PF20's eight-second sleep, this meant Prometheus reported p50 as 1.0s while Jaeger measured 8.01s. The fault was still detected, but its magnitude was only knowable through Jaeger.

### **Diagnostic content appeared in both logs and traces**

For RF12, the same failure detail showed up in two places. The fault marker, exception type, message, and stack trace were all in the auth-service logs, as expected. They also showed up in Jaeger: when the `HTTPException` was raised inside the `service_unavailable` span, OpenTelemetry attached the exception details to the span automatically.

We did not evaluate whether this overlap is useful in practice. It is left to future work.

### **Answering RQ1 and RQ2**

## RQ1 — How can fault taxonomies be mapped to observability tools?

The mapping from Chapter 3 (resource faults to metrics, slow-component faults to traces, failure events to logs) held up in practice. The three qualifications described above (detection happens across pillars, metrics can also classify faults, and the same content can appear in more than one pillar) refine the mapping rather than disprove it. It is best understood as a guide for which pillar to check first, not a strict rule.

## RQ2 — How effective are these tools at detecting and diagnosing the injected faults?

All three faults were detected and diagnosed using Prometheus, Jaeger, and Docker logs. Each fault was diagnosed most precisely by its mapped pillar:

- **PF31** was detected by Prometheus through CPU and latency metrics. Jaeger also localised it to the `cpu_hog` span, but only because the function was explicitly instrumented.
- **PF20** was localised by Jaeger to the `long_response_time` span. Prometheus detected the latency increase but capped the magnitude at one second due to its default histogram buckets.
- **RF12** was diagnosed directly through the auth-service logs. Prometheus detected the failure rate but provided no descriptive content.

In all three cases, the fault stayed contained within the affected service.

## 5.5 Limitations

The experimental design has several limitations that constrain how far the results can be generalised.

**Scale.** The experiment used three services on a single host, with one request per second per service. Real microservices systems often involve many more services, distributed across hosts, at much higher request rates. The fault containment observed here was also helped by the fact that the services shared no databases, caches, or message queues.

**Single instance per fault.** Each fault was injected once and observed for several minutes. The experiment did not measure variance across repeated injections or behaviour under concurrent multiple faults. No statistical claims can be drawn from a single observation.

**Single implementation per fault.** Each fault was implemented in one specific way. Real-world versions of the same fault categories vary considerably.

**In-request CPU hog only.** PF31 was implemented as a synchronous loop inside the request handler. An alternative form like a background CPU hog running in a separate thread would likely produce a different signature.

**Origin of source frameworks.** The two papers underpinning this project (Silva et al. (2022) and Faseeha et al. (2025)) were published in peer-reviewed but lower-prestige venues (the Brazilian Symposium on Software Engineering and IEEE Access respectively), and neither has been extensively cited or validated by the broader research community. Their findings

were treated as a useful starting point rather than authoritative claims, and the empirical results of this experiment should be read as engaging with their proposed frameworks rather than confirming established theory.

---

## 6. Conclusion

### 6.1 Summary of contribution

This project combined the fault taxonomy of Silva et al. (2022) with the observability framework of Faseeha et al. (2025) and tested the resulting mapping against three injected faults – PF31 (CPU Hog), PF20 (Long Response Time), and RF12 (Service Unavailable) – in a small FastAPI-based microservices stack instrumented with Prometheus and Jaeger.

**RQ1** was answered by the mapping presented in Chapter 3 and confirmed by the experiments: each fault was diagnosed most directly by its mapped pillar. **RQ2** was answered by the experiments themselves: all three faults were detected and diagnosed using the Prometheus/Jaeger/Docker-logs stack, with diagnostic precision highest when each fault was inspected through its naturally mapped pillar.

The more interesting finding, however, lies in the overlap between pillars. The mapping is a useful starting point, but the experiments suggest that a one-tool-per-fault approach underuses what modern observability stacks can offer. Combining tools (using metrics to classify, traces to localise, and logs for descriptive context) produces a stronger diagnosis than any single tool alone. This is the practical takeaway for the reader.

### 6.2 Future work

**Background CPU hog.** The CPU Hog used here ran synchronously inside the request handler. A different approach, like a separate thread consuming CPU independently of traffic would be possible to offer a different exploration approach.

**Centralised log aggregation.** RF12 used `docker compose logs` as the logging surface. A production-realistic setup would aggregate logs into Loki or the ELK stack, allowing structured queries and easier correlation with trace data.

**Multi-service and cascading faults.** This experiment showed fault containment within isolated services. Injecting faults that propagate across services would test the framework under more realistic failure scenarios.